

Local Value Numbering

An LLVM Optimization Pass

Redundant Load Elimination, Constant Propagation & CSE

LLVM Pass Analysis

Compiler Optimizations

May 15, 2026

Outline

- 1 Overview
- 2 Redundant Load Elimination
- 3 Constant Propagation & Folding
- 4 Common Subexpression Elimination
- 5 Dead Alloca Elimination

What is Local Value Numbering (LVN)?

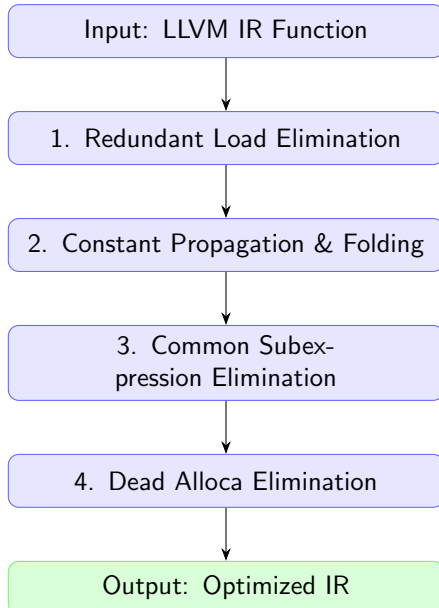
Definition

Local Value Numbering (LVN) is a compiler optimization technique that assigns a unique *value number* to expressions within a basic block to detect redundant computations.

This pass implements three sub-passes:

- ➊ **Redundant Load Elimination** — remove duplicate loads with no intervening store
- ➋ **Constant Propagation & Folding** — evaluate constant expressions at compile time
- ➌ **Common Subexpression Elimination (CSE)** — reuse already-computed values
- ➍ **A Dead Alloca Elimination** pass cleans up unused stack allocations after optimization.

Pass Architecture



Redundant Load Elimination

Problem: Multiple loads from the same pointer with no store in between.

Before:

```
store i32 %0, ptr %2, align 4  
%9 = load i32, ptr %2, align 4  
%10 = load i32, ptr %2, align 4
```

After:

```
%9 = load i32, ptr %2  
; %10 replaced by %9
```

Algorithm:

- Maintain a map: pointer $\rightarrow$ first load value
- On a new LoadInst: if pointer already in map $\Rightarrow$ replace & delete
- On a StoreInst: invalidate the pointer entry in the map

Constant Propagation & Folding

Goal: Evaluate expressions involving known constants at compile time.

getValue() Helper

Returns the integer value of an operand if it is:

- A ConstantInt (literal constant), **or**
- Tracked in the constantmap from a prior store

Operations handled:

LLVM Opcode	Operation
Add	$l + r$
Sub	$l - r$
Mul	$l \times r$
SDiv	$l \div r$
SRem	$l \bmod r$

Folded results are replaced using `replaceAllUsesWith(ConstantInt)`.

Constant Propagation — Example

Before:

```
store i32 336, ptr %6, align 4
store i32 336, ptr %7, align 4
%22 = load i32, ptr %6, align 4
%23 = load i32, ptr %7, align 4
%24 = add nsw i32 %22, %23
```

After Folding:

```
; dead stores removed
; dead loads eliminated
;%24 replaced with 672
everywhere
```

- Stores of constants update constantmap
- Subsequent loads hit the map and get replaced by the literal
- Binary ops with two known operands are folded to a single constant

Common Subexpression Elimination (CSE)

Key data structure:

$\text{DenseMap} \langle (\text{opcode}, V_L, V_R) \rightarrow \text{Value}^* \rangle$

- Each expression is a **3-tuple**: (opcode, left operand, right operand)
- For commutative ops (Add, Mul): operands are **sorted by pointer** so $a + b$ and $b + a$ hash identically
- On a cache hit $\Rightarrow$ replace current instruction with cached value and delete
- On a cache miss $\Rightarrow$ insert the tuple into the map

Constant Propagation — Example

Before:

```
%9 = load i32, ptr %2, align 4
%10 = load i32, ptr %2, align 4
%11 = mul nsw i32 %9, %10
store i32 %11, ptr %3, align 4
%12 = load i32, ptr %2, align 4
%13 = load i32, ptr %2, align 4
%14 = mul nsw i32 %12, %13
```

Figure: Enter Caption

After Folding:

```
; dead loads and stores are
removed by
; constant propagation
; %14 is replaced by %11
everywhere
```

Commutativity

```
if (hashl > hashr && (op->getOpcode() == Instruction::Add ||
                    op->getOpcode() == Instruction::Mul)) {
    std::swap(hashl,
              hashr); // this is to maintain commutativity , so a+b and
                    // b+a are always stored in the same order
```

Dead Alloca Elimination

Why?

After constant propagation and CSE, some `alloca` instructions may have no remaining uses — they become **dead allocas**.

Algorithm:

- 1 Collect all `AllocaInst` instructions into a map
- 2 Traverse the function; for every `StoreInst`, **remove** the destination pointer from the map (it is being used)
- 3 All entries remaining in the map were never stored to — **erase them**

Before:

```
%a = alloca i32  
; no store to %a  
; %a never used
```

After:

```
; %a alloca removed
```